DevOps Laboratory (25DC2003)

Detailed Study Notes

Experiments 1, 2, 3, 4, 6 and 7

Exp	Title
1	Git Essentials - Commit, Push, Merge and Pull Requests
2	Flask App Containerization and Docker Hub Deployment
3	CI Pipeline for a Python App Using GitHub Actions and Docker Hub
4	Jenkins Pipeline Deployment of a Static Website Using Docker
6	Managing Kubernetes Resources Using kubectl CLI
7	Kubernetes Deployment and Service for a Python App from Docker Hub

Every experiment contains: Aim, Introduction, Description, a detailed step-by-step Procedure (with the command, what it does, and what you should see), the content of every file, and a short Result statement.

Exp No. 1

Git Essentials - Commit, Push, Merge and Pull Requests

Version control with Git and GitHub

Aim

To demonstrate pushing changes to a GitHub repository, covering Git setup, managing repository visibility (private/public), and executing basic Git operations such as commit, push, branch, merge, pull and pull request.

Introduction

Git is a **Distributed Version Control System (DVCS)**. Version control records every change made to a project so that you can go back to any earlier version, compare changes, and work with other people without overwriting their work. In a distributed system every developer holds a complete copy of the repository and its history on their own computer, so most operations (commit, log, branch) work offline and are very fast. **GitHub** is a cloud platform that hosts Git repositories and adds collaboration features such as pull requests, issues, code review and GitHub Actions.

Key terms

Term	Meaning
Repository (repo)	A project folder tracked by Git. The hidden .git folder stores the full history.
Working directory	The files you are currently editing on disk.
Staging area (index)	A holding area where you choose which changes will go into the next commit (git add).
Commit	A saved snapshot of the staged changes with a message and a unique ID (hash).
Branch	An independent line of development (default branch is main). Lets you work on a feature safely.
Merge	Combines the history of one branch into another.
Remote / origin	The copy of the repository hosted on a server (GitHub). 'origin' is the usual name.
Push / Pull / Clone	Push uploads commits to the remote; pull downloads and merges remote changes; clone makes a local copy of a remote repo.
Pull request (PR)	A request on GitHub to merge one branch into another, so others can review the changes first.

Description

The experiment has four parts: (1) install Git on the computer, (2) understand what happens when a repository is made private or public, (3) learn how to change the visibility of a repository on GitHub, and (4) make changes in a local repository, commit them and push them to the remote repository on GitHub.

Making a repository private

- Public forks are detached from the repository and moved to a new network; public forks are NOT made private.
- On GitHub Free, some features become unavailable. A published GitHub Pages site is automatically unpublished (remove or update custom-domain DNS records first to avoid a domain takeover).
- The repository is no longer part of the GitHub Archive Program.
- GitHub Advanced Security features such as code scanning stop working.

Making a repository public

- Private forks are detached and become standalone private repositories.
- Used when turning a project into an open-source project; anyone can view and clone it.
- The repository's community profile can be viewed to check best practices for contributors.
- The repository automatically gains access to GitHub Advanced Security features.

Procedure

Step 1: Check whether Git is already installed

Open Command Prompt / PowerShell / terminal and check the version. If a version number is printed, Git is installed (update it if it is old); if the command is not recognised, install it.

```
git --version
```

Expected output / observation: git version 2.37.3.windows.1 (the number can differ).

Step 2: Download and install Git on Windows

Go to https://git-scm.com/download/win; the download starts automatically. Choose **64-bit Git for Windows Setup** (about 50 MB). Double-click the downloaded .exe and click Next through the screens. Two screens matter: (a) **Default editor** - the default is Vim, which is hard for beginners, so pick Visual Studio Code, Notepad++ or another plain text editor; (b) **Adjusting your PATH environment** - select **'Git from the command line and also from 3rd-party software'** (recommended), which lets you use git from Git Bash, Command Prompt and PowerShell. Accept the defaults for the remaining screens and click Install (the default selection downloads/installs about 270 MB). Linux: *sudo apt install git*. macOS: *brew install git*. Alternative on Windows: the Git Chocolatey package for an automated install.

Expected output / observation: After installation, reopen the terminal and run git --version again to confirm.

Step 3: Configure your identity (one time)

Git stores your name and e-mail in every commit. Use the same e-mail as your GitHub account.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
```

Expected output / observation: The last command lists user.name and user.email along with other settings.

Step 4: Create an empty repository on GitHub

Sign in at github.com, click the '+' icon (top right) and choose **New repository**. Enter a repository name (for example git-lab), choose **Public** or **Private**, and leave 'Add a README' unticked so that the remote starts empty (this avoids conflicts on the first push). Click **Create repository** and copy the HTTPS URL, e.g. https://github.com/<user>/git-lab.git.

Step 5: Create the local project folder and a file

Create a folder and a sample index.html (content is given in the Files section at the end).

```
mkdir git-lab
cd git-lab
# create index.html using any editor
```

Step 6: Initialise a local Git repository

git init creates the hidden .git directory that stores all history of the project.

```
git init
```

Expected output / observation: Initialized empty Git repository in .../git-lab/.git/

Step 7: Check the status

git status shows the state of the working directory and staging area. A new file is 'untracked'.

```
git status
```

Expected output / observation: 'Untracked files: index.html' is displayed (in red in most terminals).

Step 8: Stage the file

git add moves the change to the staging area. Use *git add .* or *git add -A* to stage all modified and new files.

```
git add index.html
git status
```

Expected output / observation: Now it says 'Changes to be committed: new file: index.html' (green).

Step 9: Commit the change

A commit permanently records the staged snapshot with a descriptive message in the local history.

```
git commit -m "Initial commit of the project"
```

Expected output / observation: [master (root-commit) 3fa9c21] Initial commit of the project - 1 file changed, 8 insertions(+).

Step 10: View the history

git log lists the commits of the current branch with author, date and message.

```
git log
```

```
git log --oneline
```

Step 11: Rename the branch and connect to GitHub

Rename the default branch to main, then link the local repository to the GitHub repository using the name 'origin'. Verify the link with git remote -v.

```
git branch -M main
git remote add origin https://github.com/<user>/git-lab.git
git remote -v
```

Expected output / observation: Two lines (fetch) and (push) showing the origin URL.

Step 12: Push the commits to GitHub

git push uploads local commits to the remote. -u sets origin/main as the upstream so later you can just type git push. GitHub asks you to sign in (browser window) or to use a Personal Access Token as the password.

```
git push -u origin main
```

Expected output / observation: 'Writing objects: 100%' and 'Branch main set up to track remote branch main from origin'. Refresh the repository page on GitHub: index.html is now visible.

Step 13: Create a feature branch and make a change

Branches let you develop a feature without disturbing main. git checkout -b creates and switches to the new branch in one command (git branch lists branches, the current one has a star).

```
git checkout -b feature/new-feature
# edit index.html (add a new line or paragraph)
git add .
git commit -m "Add new feature"
git branch
```

Expected output / observation: Output of git branch: '* feature/new-feature' and 'main'.

Step 14: Merge the feature branch into main (local merge)

Switch back to main and merge. If main has not changed since the branch was created, Git does a 'fast-forward' merge. Then push main and delete the finished branch.

```
git checkout main
git merge feature/new-feature
git push origin main
git branch -d feature/new-feature
```

Expected output / observation: 'Updating 3fa9c21..8b12d4e Fast-forward index.html | 1 +'.

Note: If two branches change the same line, Git reports a merge conflict. Open the file, look for the markers <<<<<<<, ======= and >>>>>>>, keep the correct text, delete the markers, then git add and git commit.

Step 15: Create a Pull Request (PR) on GitHub

A PR is the team way of merging. Create another branch, commit, and push it: *git checkout -b feature/pr-demo*, edit, *git add .*, *git commit -m 'PR demo'*, *git push -u origin feature/pr-demo*. On GitHub a yellow banner 'Compare & pull request' appears - click it. Check that base = main and compare = feature/pr-demo, write a title and description, and click **Create pull request**. Open the **Files changed** tab to review the differences, then click **Merge pull request** and **Confirm merge**. Optionally click 'Delete branch'.

Expected output / observation: The PR shows the purple 'Merged' badge and main on GitHub now contains the new commit.

Step 16: Pull the merged changes to your computer

git pull fetches the remote changes and merges them into the local branch, keeping your computer in sync with GitHub.

```
git checkout main
git pull origin main
```

Expected output / observation: 'Updating ... Fast-forward' and the file shows the PR's change.

Step 17: Change the repository visibility

(1) On GitHub.com open the main page of the repository. (2) Click **Settings** under the repo name (if hidden, open the '...' dropdown). (3) Scroll to the **Danger Zone** and click **Change visibility** next to 'Change repository visibility'. (4) Select Make private or Make public. (5) Type the repository name to confirm you are changing the correct repository. (6) Click **I understand, change repository visibility**.

Expected output / observation: A 'Private' or 'Public' badge appears beside the repository name. A private repo shows 404 to people who are not logged in/collaborators.

Files and content

index.html (sample file used for the commits)

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Git Project</title>
</head>
<body>
  <h1>Hello Git!</h1>
  <p>This file is tracked by Git.</p>
</body>
</html>
```

Added line on the feature branch (append inside <body>)

```
  <p>New feature added on feature/new-feature branch.</p>
```

Result

Git was installed and configured, a local repository was created, files were committed, branches were created and merged, changes were pushed to and pulled from GitHub, a pull request was merged, and the repository visibility was changed between private and public.

Exp No. 2

Flask App Containerization and Docker Hub Deployment

Python Flask + Docker + Docker Hub

Aim

To design and create a Python Flask web application, containerize it using Docker, test the container locally on specified ports, and deploy (push) the container image to Docker Hub for public accessibility and cross-platform execution.

Introduction

Containerization is an operating-system-level virtualization technique that runs an application in an isolated user space called a **container**. The container bundles the application code, runtime, libraries and settings, so it runs the same way on any machine - this solves the 'it works on my machine' problem. Unlike virtual machines, containers share the host OS kernel, so they start in seconds and use much less memory. Docker is the most popular container platform and Docker Hub is its public registry.

Key terms

Term	Meaning
Flask	A lightweight Python micro web framework for building web apps and REST APIs.
Dockerfile	A text file of ordered instructions used to build an image automatically.
Image	A read-only, standalone package containing code, runtime, libraries and settings.
Container	A running instance of an image, isolated from the host and other containers.
Port binding (-p host:container)	Forwards traffic from a host port to the port the app listens on inside the container.
Docker Hub	Cloud registry used to store, share and distribute images.
Tag	A label for an image version, e.g. latest; the name must start with your Docker Hub username before pushing.

Description

A Flask app that shows a styled HTML card with a gradient background is written. A Dockerfile based on python:3.9-slim packages it. The image *flask-docker-app* is built and run on port 5000. It is then tagged as *klassy01/flask-docker-app:latest*, pushed to Docker Hub, and finally pulled and run again on port 5050 to prove the image is portable.

Procedure

Step 1: Check the prerequisites

Install Docker Desktop (Windows/macOS) or Docker Engine (Linux), start it, and make sure the whale icon says Docker is running. Create a free Docker Hub account (username klassy01 in this record).

```
docker --version
docker info
```

Expected output / observation: Docker version 2x.x.x, build ... and server details (an error here means Docker is not running).

Step 2: Create the project folder

Create a directory with three files: app.py, requirements.txt and Dockerfile (contents are at the end).

```
mkdir flask-docker-app
cd flask-docker-app

flask-docker-app/
 |-- app.py
 |-- requirements.txt
 |-- Dockerfile
```

Step 3: Write the Flask application (app.py)

The app creates a Flask object, defines an HTML template string with CSS (gradient background and a translucent card), maps the route '/' to a function that returns the rendered template, and starts the server with **host='0.0.0.0'** and port 5000. host 0.0.0.0 is essential: if it were 127.0.0.1 the app would be unreachable from outside the container.

Step 4: List the dependencies (requirements.txt)

requirements.txt contains one line, Flask==2.2.5. Pinning the version makes builds repeatable.

Step 5: Write the Dockerfile

Each instruction creates a layer of the image. The meaning of every line is in the table below.

Instruction	Purpose
FROM python:3.9-slim	Start from a small official Python 3.9 base image.
WORKDIR /app	Set the working directory inside the container.
COPY requirements.txt .	Copy only the dependency list first so Docker can cache the pip layer.
RUN pip install -r requirements.txt	Install Flask while building the image.
COPY . .	Copy the rest of the source code into /app.
EXPOSE 5000	Documents that the container listens on port 5000.
CMD ["python", "app.py"]	Command that runs when the container starts.

Step 6: Build the Docker image

Run in the folder that contains the Dockerfile. '-t' gives the image a name (tag); the dot means 'use the current directory as build context'.

```
docker build -t flask-docker-app .
docker images
```

Expected output / observation: Build steps [1/5] ... [5/5] complete, 'naming to docker.io/library/flask-docker-app:latest'. docker images lists flask-docker-app with its size.

Step 7: Run the container and test locally

'-p 5000:5000' maps host port 5000 to container port 5000. Keep the terminal open (it shows logs) and browse to http://localhost:5000.

```
docker run -p 5000:5000 flask-docker-app
```

Expected output / observation: Logs: 'Running on http://0.0.0.0:5000' and '172.17.0.1 - - GET / HTTP/1.1 200'. The browser shows 'Welcome to Flask + Docker!' on a blue gradient card. Press Ctrl+C to stop.

Note: If port 5000 is already used, run docker run -p 5050:5000 flask-docker-app and open http://localhost:5050 (host port changes, container port stays 5000).

Step 8: (Optional) Run in the background and manage the container

-d runs detached and --name gives a friendly name. These commands show, stop and remove it.

```
docker run -d --name flask-web -p 5000:5000 flask-docker-app
docker ps
docker ps -a
docker logs flask-web
docker stop flask-web
docker rm flask-web
```

Step 9: Log in to Docker Hub

Authenticate the Docker CLI with your Docker Hub username and password (or access token).

```
docker login
```

Expected output / observation: 'Login Succeeded'.

Step 10: Tag the image with your Docker Hub username

Docker Hub only accepts images named <username>/<repo>:<tag>. Tagging creates a second name for the same image (same IMAGE ID).

```
docker tag flask-docker-app klassy01/flask-docker-app:latest
```

```
docker images
```

Expected output / observation: Two rows with the same IMAGE ID: flask-docker-app and klassy01/flask-docker-app.

Step 11: Push the image to Docker Hub

Uploads the image layers; layers already on Docker Hub show 'Layer already exists'.

```
docker push klassy01/flask-docker-app:latest
```

Expected output / observation: Lines 'Pushed' / 'Layer already exists' and finally 'latest: digest: sha256:... size: ...'. Open hub.docker.com, repository klassy01/flask-docker-app, to see the image listed as public.

Step 12: Pull and run the image to prove portability

On any machine with Docker (or after deleting the local copy with docker rmi), pull the image from Docker Hub and run it on a different host port.

```
docker pull klassy01/flask-docker-app:latest
docker run -p 5050:5000 klassy01/flask-docker-app:latest
```

Expected output / observation: Image is downloaded and the same web page opens at http://localhost:5050.

Step 13: Clean up

Remove stopped containers and (optionally) images to free space.

```
docker ps -a
docker rm <container_id>
docker rmi flask-docker-app
```

Common error	Cause / fix
Cannot connect to the Docker daemon	Docker Desktop is not running - start it.
port is already allocated	Another program uses that host port - use -p 5050:5000.
denied: requested access to the resource is denied (push)	Not logged in, or the image name does not start with your username - re-tag.
Page not reachable on localhost	app.run() host is not 0.0.0.0, or -p mapping is wrong.

Files and content

app.py

```
from flask import Flask, render_template_string
app = Flask(__name__)

HTML_TEMPLATE = """
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Flask Docker App</title>
<style>
body { background: linear-gradient(to right, #00c6ff, #0072ff);
       color: white; font-family: 'Segoe UI', sans-serif;
       text-align: center; padding-top: 100px; }
h1 { font-size: 3em; margin-bottom: 0.2em; }
p  { font-size: 1.5em; }
.card { background: rgba(255,255,255,0.1); padding: 2em;
        border-radius: 20px; box-shadow: 0 0 20px rgba(0,0,0,0.3);
        width: 50%; margin: auto; }
</style>
</head>
<body>
<div class="card">
<h1>Welcome to Flask + Docker!</h1>
<p>This is a vibrant web page running inside a Docker container.</p>
<p>Change the code -> Rebuild -> Restart Docker to see updates!</p>
</div>
</body>
</html>
"""

@app.route('/')
def home():
    return render_template_string(HTML_TEMPLATE)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
Flask==2.2.5
```

Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Result

The Flask web application was designed, containerized with Docker, verified locally on ports 5000/5050, tagged and pushed to Docker Hub as klassy01/flask-docker-app:latest, and successfully pulled and run again, demonstrating container portability.

Exp No. 3

CI Pipeline for a Python App Using GitHub Actions and Docker Hub

pytest + Docker + GitHub Actions (CI)

Aim

To create a simple Python application and configure a GitHub Actions workflow that automatically runs tests with pytest, builds a Docker image and pushes the image to Docker Hub, with an optional release tag (v1.0.0).

Introduction

Continuous Integration (CI) is the practice of automatically building and testing code every time it changes, so that bugs are found within minutes instead of at release time. **GitHub Actions** is GitHub's built-in CI/CD service: a *workflow* written in YAML under .github/workflows/ is started by events such as push, pull request or tag creation, and runs on a GitHub-hosted *runner* (a fresh virtual machine, here ubuntu-latest).

Key terms

Term	Meaning
Workflow / Job / Step	A workflow contains jobs; a job runs on a runner and consists of ordered steps (uses: an action, or run: a command).
pytest	Python testing framework; finds functions named test_* and runs them.
GitHub Secrets	Encrypted variables (DOCKERHUB_USERNAME, DOCKERHUB_TOKEN) read as ${{ secrets.NAME }}; never stored in code.
Docker Buildx	Docker's modern build engine used by the build-push action.
metadata-action	Generates image tags: latest, sha-<commit>, and the git tag name.
Access token	A revocable key used instead of the Docker Hub password.
Quality gate	Image is pushed only if the tests pass.

Description

A tiny Python app with an *add()* function and a pytest test is stored in a GitHub repository. On every push to main (or pull request to main, or tag) the workflow: checks out the code, installs Python and dependencies, runs pytest, prepares image tags, logs in to Docker Hub using the secrets, then builds and pushes *klassy01/python-ci-lab*. Pushing the tag **v1.0.0** additionally publishes an image tagged v1.0.0.

Procedure

Step 1: Create the project folder

```
mkdir python-ci-docker-lab
cd python-ci-docker-lab
mkdir tests
```

Step 2: Create the application and test files

Create app.py, tests/test_app.py, requirements.txt, Dockerfile and .gitignore with the contents given in the Files section. app.py defines add(a, b); the test imports add and asserts add(2,3)==5 and add(-1,1)==0. requirements.txt pins pytest==8.3.2.

Step 3: Create the workflow file

GitHub only looks for workflows in this exact folder. Save the YAML (see Files section) as ci-dockerhub.yml. YAML is indentation-sensitive - use spaces, never tabs.

```
mkdir -p .github/workflows
```

Workflow part	What it does
on: push / tags / pull_request	Triggers on pushes to main, any tag (e.g. v1.0.0) and PRs to main.
runs-on: ubuntu-latest	Runs the job on a GitHub-hosted Ubuntu VM.

Workflow part	What it does
actions/checkout@v4	Downloads the repository code onto the runner.
actions/setup-python@v5	Installs Python 3.11.
Install deps	Upgrades pip and installs requirements.txt (pytest).
Run tests (pytest -q)	If any test fails the job stops here and nothing is pushed.
docker/metadata-action@v5	Creates tags: latest (on default branch), sha-xxxxxxx, and the git tag.
setup-qemu / setup-buildx	Prepare the Docker build environment.
docker/login-action@v3	Logs in to Docker Hub using the two secrets.
docker/build-push-action@v6	Builds the Dockerfile and pushes with the generated tags (platform linux/amd64).

Step 4: Test locally (optional but recommended)

Verify the code and the image work before using CI.

```
python app.py
pytest -q
docker build -t klassy01/python-ci-lab:local .
docker run --rm klassy01/python-ci-lab:local
```

Expected output / observation: python app.py prints 'Hello from Python CI Lab!' and '2 + 3 = 5'. pytest shows '1 passed in 0.xxs'. docker run prints the same two lines and the container is removed (--rm).

Step 5: Create a Docker Hub access token

Log in at hub.docker.com, open Account Settings, Security (Personal access tokens), click **Generate new token**, give it a name, choose Read & Write permission and copy the token immediately (it is shown only once).

Step 6: Create the GitHub repository and add secrets

Create an empty repository named python-ci-docker-lab on GitHub. Open **Settings > Secrets and variables > Actions > New repository secret** and add two secrets: **DOCKERHUB_USERNAME** (klassy01) and **DOCKERHUB_TOKEN** (the token). Adding the secrets before the first push avoids a failed first run.

Step 7: Initialise Git and push - this triggers the workflow

```
git init
git add .
git commit -m "init lab"
git branch -M main
git remote add origin https://github.com/<your-username>/python-ci-docker-lab.git
git push -u origin main
```

Expected output / observation: Commit shows '6 files changed' (.github/workflows/ci-dockerhub.yml, .gitignore, Dockerfile, app.py, requirements.txt, tests/test_app.py).

Note: If the first run failed because secrets were missing, open the run in the Actions tab and click **Re-run all jobs** after adding them.

Step 8: Watch the CI run

Open the repository, click the **Actions** tab, select the latest run 'ci-dockerhub' and click the job build-test-push. Expand each step to read its log.

Expected output / observation: Green ticks for: Set up job, Checkout, Set up Python (3.11), Install deps, Run tests ('1 passed'), Docker meta, QEMU, Buildx, Login to Docker Hub, Build and push ('Pushing ... latest done'), Complete job. Total time about 1-2 minutes.

Step 9: Verify the image on Docker Hub

Open hub.docker.com/r/klassy01/python-ci-lab, tab Tags. Or pull it.

```
docker pull klassy01/python-ci-lab:latest
docker run --rm klassy01/python-ci-lab:latest
```

Expected output / observation: Tags latest and sha-<7-char commit id> are present; the container prints the Hello message.

Step 10: Release with a version tag (optional)

A git tag marks a release commit. Pushing it triggers the workflow again and the metadata step adds the tag name as an image tag.

```
git tag v1.0.0
git push origin v1.0.0
docker images klassy01/python-ci-lab
```

Expected output / observation: A new image tag v1.0.0 appears on Docker Hub (same image content as latest).

Step 11: (Optional) Prove the quality gate

Change the test to assert add(2, 3) == 6, commit and push. The Run tests step fails (red cross) and the Build and push steps are skipped, so no broken image reaches Docker Hub. Revert the change to make the pipeline green again.

Files and content

app.py

```
def add(a, b):
    return a + b

if __name__ == "__main__":
    print("Hello from Python CI Lab!")
    print("2 + 3 =", add(2, 3))
```

tests/test_app.py

```
from app import add

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
```

requirements.txt

```
pytest==8.3.2
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

.gitignore

```
__pycache__/
.venv/
.pytest_cache/
.DS_Store
*.pyc
```

.github/workflows/ci-dockerhub.yml

```
name: ci-dockerhub
on:
  push:
    branches: [ "main" ]
    tags: [ "*" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-test-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install deps
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
```

```
- name: Run tests
  run: pytest -q

- name: Docker meta
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${{ secrets.DOCKERHUB_USERNAME }}/python-ci-lab
    tags: |
      type=raw,value=latest,enable={{is_default_branch}}
      type=sha,prefix=sha-,format=short
      type=ref,event=tag

- name: Set up QEMU
  uses: docker/setup-qemu-action@v3

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Login to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKERHUB_USERNAME }}
    password: ${{ secrets.DOCKERHUB_TOKEN }}

- name: Build and push
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    platforms: linux/amd64
```

Result

A Python application with pytest tests was committed to GitHub; a GitHub Actions workflow automatically ran the tests, built the Docker image and pushed it to Docker Hub as klassy01/python-ci-lab. The release tag v1.0.0 also produced a versioned image, demonstrating a working CI pipeline.

Exp No. 4

Jenkins Pipeline Deployment of a Static Website Using Docker

Jenkins in Docker Compose + Declarative Pipeline + HTML Publisher

Aim

To run Jenkins inside Docker using Docker Compose and to create a Jenkins pipeline that generates a static website, publishes it inside Jenkins using the HTML Publisher plugin, and archives the build artifacts.

Introduction

Jenkins is an open-source automation server used for continuous integration and continuous delivery: it builds, tests and deploys software through jobs and pipelines. Running Jenkins in a Docker container makes the CI server disposable and reproducible - it can be started anywhere with one command and upgraded by changing the image tag. A **pipeline** is the build process written as code (a Jenkinsfile in Groovy), so it can be version-controlled and reviewed like application code.

Key terms

Term	Meaning
Docker Compose	Tool that defines and runs containers from a docker-compose.yml file (docker compose up/down).
Docker volume (jenkins_home)	Persistent storage; Jenkins jobs, plugins and settings survive container restarts.
Pipeline / Stage / Step	A pipeline is a series of stages; a stage is a named phase; steps are the commands inside.
agent any	Run the pipeline on any available Jenkins agent.
writeFile	Pipeline step that creates a file in the workspace from text.
publishHTML	HTML Publisher plugin step that publishes a folder as a report linked from the job.
archiveArtifacts	Stores build output so it can be downloaded later; fingerprint tracks files across builds.
post { always { } }	Steps that run after the stages whether the build passes or fails.
Initial admin password	One-time password read from the container to unlock Jenkins the first time.

Description

Jenkins (image jenkins/jenkins:lts) is started with Docker Compose on port 8080 (50000 is for agents). A pipeline job *karunya-site-pipeline* has two stages: **Generate site** writes index.html and assets/style.css into a site/ folder in the workspace (the year in the footer is taken from the system date); **Publish site** publishes the folder with the HTML Publisher plugin. A post action archives site/**. The result is a Karunya University web page viewable inside Jenkins plus downloadable artifacts.

Procedure

Step 1: Prerequisites

Docker Desktop (with Docker Compose) must be running and ports 8080 and 50000 must be free.

```
docker --version
docker compose version
```

Step 2: Create the lab folder

```
mkdir ~/jenkins-static-site && cd ~/jenkins-static-site
# Windows PowerShell: mkdir jenkins-static-site ; cd jenkins-static-site
```

Step 3: Create docker-compose.yml

Create the file with the content given at the end. Meaning of the keys: **image** - Jenkins LTS image; **container_name** - fixed name 'jenkins'; **restart: unless-stopped** - auto restart; **ports** - 8080 web UI and 50000 agent port; **volumes** - jenkins_home persists data and docker.sock lets Jenkins talk to the host Docker.

Step 4: Start Jenkins

-d runs the container in the background. The first start downloads the image (a few minutes).

```
docker compose up -d
docker ps
```

Expected output / observation: 'Network jenkins-static-site_default Created' and 'Container jenkins Started'. docker ps shows the jenkins container with ports 0.0.0.0:8080->8080/tcp.

Step 5: Get the initial admin password

Read the one-time password from inside the container (it is also printed in docker compose logs jenkins).

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Expected output / observation: A 32-character hex string, e.g. 7f8a9e01b2c3d4e5f67890123456789a.

Step 6: Complete the Jenkins setup wizard

Open http://localhost:8080 in a browser, paste the password in 'Unlock Jenkins', click Continue, choose **Install suggested plugins** and wait, then create the first admin user (username, password, name, e-mail) and keep the default Jenkins URL. Click Start using Jenkins.

Expected output / observation: The Jenkins dashboard appears.

Step 7: Install the HTML Publisher plugin

Go to **Manage Jenkins > Plugins > Available plugins**, search for 'HTML Publisher', tick it and click **Install**. Tick 'Restart Jenkins when installation is complete' if offered, and log in again.

Expected output / observation: The plugin shows under Installed plugins. Without it, the pipeline fails at publishHTML.

Step 8: Create the Pipeline job

On the dashboard click **New Item**, enter the name **karunya-site-pipeline**, choose **Pipeline** and click OK. In the Pipeline section set Definition = **Pipeline script**.

Step 9: Paste the pipeline script and save

Paste the Jenkinsfile given in the Files section into the Script box and click Save. What it does: stage 1 gets the year with the shell command date +%Y, removes any old site folder, creates site/assets, and writes index.html and style.css; stage 2 runs publishHTML with reportDir 'site' and reportFiles 'index.html' (reportName becomes the link text); the post block archives site/**.

Step 10: Run the build

Click **Build Now**. A build #1 appears under Build History; click it and open **Console Output** to follow the log.

Expected output / observation: Stage View shows Generate site and Publish site in green, and the log ends with 'Finished: SUCCESS'. Build time is only a few seconds.

Step 11: View the published website inside Jenkins

On the build (or job) page click **'Karunya University - Simple Site'** in the left menu.

Expected output / observation: A blue header 'Karunya University', the tagline 'Values - Knowledge - Service', a card 'About Karunya University' and a footer with the current year 'Generated by Jenkins'.

Step 12: Check the archived artifacts

On the build page under **Build Artifacts** download site/index.html and site/assets/style.css (or the whole site as a zip) and open index.html locally in a browser.

Step 13: Clean up (optional)

docker compose down stops and removes the container but keeps the jenkins_home volume (data survives). Add -v only if you also want to delete the data.

```
docker compose down
```

Problem	Fix
Port 8080 already in use	Stop the other program, or change the mapping to '9090:8080' and open localhost:9090.
No such DSL method 'publishHTML'	HTML Publisher plugin is not installed or Jenkins was not restarted.
Cannot find password	Run docker compose logs jenkins and look for the generated password block.

Problem	Fix
Website shows unstyled	The report is served with a strict Content Security Policy; the CSS may be blocked (Exp 5 explains how to relax it).

Files and content

docker-compose.yml

```
version: '3.8'
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: unless-stopped
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
volumes:
  jenkins_home:
    driver: local
```

Jenkinsfile (pasted in 'Pipeline script')

```
pipeline {
  agent any
  stages {
    stage('Generate site') {
      steps {
        script {
          def year = sh(script: 'date +%Y', returnStdout: true).trim()
          sh 'rm -rf site || true'
          sh 'mkdir -p site/assets'
          writeFile file: 'site/index.html', text: """<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<title>Karunya University - Simple Site</title>
<link rel="stylesheet" href="assets/style.css"/>
</head>
<body>
<header class="site-header">
<div class="container">
<h1>Karunya University</h1>
<p class="tagline">Values - Knowledge - Service</p>
</div>
</header>
<main class="container">
<section class="card">
<h2>About Karunya University</h2>
<p>Karunya Institute of Technology and Sciences is a leading centre for higher education in India.</p>
</section>
</main>
<footer class="site-footer container">
<p>&copy; ${year} Karunya University - Generated by Jenkins</p>
</footer>
</body>
</html>
"""
          writeFile file: 'site/assets/style.css', text: """
body { font-family: Arial, sans-serif; margin:0; }
.container { max-width:800px; margin:0 auto; padding:20px; }
.site-header { background:#0b3d91; color:#fff; padding:20px; }
.tagline { font-size:14px; opacity:0.8; }
.card { background:#fff; border:1px solid #ddd; padding:15px; margin:15px 0; border-radius:6px; }
.site-footer { text-align:center; font-size:12px; color:#555; margin-top:20px; }
"""
        }
      }
    }
    stage('Publish site') {
      steps {
        publishHTML([
          allowMissing: false,
          alwaysLinkToLastBuild: true,
          keepAll: true,
          reportDir: 'site',
          reportFiles: 'index.html',
          reportName: 'Karunya University - Simple Site'
```

```
        ])
      }
    }
  }
  post {
    always {
      archiveArtifacts artifacts: 'site/**', fingerprint: true
    }
  }
}
```

Result

Jenkins was run inside Docker using Docker Compose. A pipeline job generated a static Karunya University website, published it inside Jenkins with the HTML Publisher plugin, and archived the site folder as a build artifact.

Exp No. 6

Managing Kubernetes Resources Using kubectl CLI

Docker + kubectl + Minikube

Aim

To install and configure the Kubernetes tool chain (Docker, kubectl and Minikube) and to manage basic Kubernetes resources using the kubectl command-line interface: creating Pods and Deployments, scaling, rolling updates and rollbacks, exposing a Deployment as a Service, and cleaning up resources.

Introduction

Kubernetes (K8s) is an open-source container orchestration platform that schedules, scales and heals containerized applications across a cluster of machines. It is the industry standard for running containers in production. **kubectl** is the command-line tool that talks to the cluster's API server. **Minikube** creates a single-node Kubernetes cluster on your own computer (here inside Docker) for learning and testing. A Deployment keeps the desired number of Pods running, restarts failed Pods and supports zero-downtime rolling updates and rollbacks.

Key terms

Term	Meaning
Cluster / Node	A set of machines (nodes) running containers; Minikube makes a one-node cluster.
Pod	Smallest deployable unit; one or more containers sharing network and storage.
Deployment	Controller that manages identical Pods through a ReplicaSet; enables scaling, updates and rollbacks.
ReplicaSet	Ensures the specified number of Pod replicas are running.
Service	Stable network endpoint for a group of Pods. Types: ClusterIP, NodePort, LoadBalancer.
NodePort	Service type that opens a static port (30000-32767) on every node.
Labels / Selectors	Key-value tags (app: nginx) used to group and select resources.
Manifest (YAML)	Declarative file describing the desired state of a resource.

Description

After Docker, kubectl and Minikube are installed and the node is Ready, a Pod is created from a YAML manifest and opened using port-forwarding. Then a Deployment *my-nginx* is created, scaled to 3 replicas, updated to nginx:1.25 (rolling update) with the ability to roll back, exposed through a NodePort Service and opened in the browser. Finally all resources and the cluster are removed.

Procedure

Step 1: Install Docker

Download Docker Desktop from Docker's website, enable the WSL2 backend (Windows), start Docker Desktop and wait until it says Docker is running.

```
docker --version
```

Expected output / observation: Docker version 28.3.2, build 578ccf6 (number may differ).

Step 2: Install kubectl

Open PowerShell **as Administrator** (Chocolatey needs admin rights) and install the Kubernetes CLI.

```
choco install kubernetes-cli
kubectl version --client
```

Expected output / observation: Client Version: v1.32.2 and Kustomize Version: v5.5.0.

Step 3: Install Minikube

```
choco install minikube
minikube version
```

Step 4: Start the local cluster

--driver=docker runs the Kubernetes node as a Docker container. The first start downloads images and takes a few minutes. Make sure Docker Desktop is running first.

```
minikube start --driver=docker
minikube status
kubectl get nodes
```

Expected output / observation: minikube status shows host, kubelet and apiserver 'Running' and kubeconfig 'Configured'. kubectl get nodes: NAME minikube, STATUS **Ready**, ROLES control-plane, VERSION v1.32.0. Do not continue until the node is Ready.

Step 5: Create a Pod from a manifest

Create nginx-pod.yaml (see Files section) and apply it. 'apply -f' creates the resource declared in the file.

```
kubectl apply -f nginx-pod.yaml
kubectl get pods
```

Expected output / observation: pod/nginx-pod created; then nginx-pod READY 1/1, STATUS Running (it may show ContainerCreating for a few seconds while the image is pulled).

Step 6: Inspect the Pod

describe shows the node, IP address, container image, ports and the Events list (useful for debugging ImagePullBackOff or CrashLoopBackOff).

```
kubectl describe pod nginx-pod
kubectl logs nginx-pod
```

Step 7: Access the Pod using port-forward

Forwards local port 8080 to port 80 of the Pod. Keep this terminal open and browse to http://localhost:8080.

```
kubectl port-forward pod/nginx-pod 8080:80
```

Expected output / observation: 'Forwarding from 127.0.0.1:8080 -> 80'; the browser shows 'Welcome to nginx!'.

Step 8: Delete the Pod

Press Ctrl+C to stop port-forwarding, then delete the Pod. A bare Pod is not recreated after deletion.

```
kubectl delete pod nginx-pod
kubectl get pods
```

Expected output / observation: pod "nginx-pod" deleted; 'No resources found in default namespace.'

Step 9: Create a Deployment

A Deployment creates a ReplicaSet which creates the Pods. The Pods get the label app=my-nginx automatically, and -l filters Pods by label.

```
kubectl create deployment my-nginx --image=nginx
kubectl get deployments
kubectl get pods -l app=my-nginx
```

Expected output / observation: deployment.apps/my-nginx created; READY 1/1; one Pod named my-nginx-5b584c864b-xxxxx Running.

Step 10: Scale the Deployment

Change the number of replicas to 3. Kubernetes starts two more Pods. Deleting one Pod (kubectl delete pod <name>) shows self-healing: a new Pod is created at once.

```
kubectl scale deployment my-nginx --replicas=3
kubectl get deployments
kubectl get pods
```

Expected output / observation: deployment.apps/my-nginx scaled; READY 3/3 and three Pods Running.

Step 11: Perform a rolling update

Update the container image to a fixed version. The format is deployment/<name> <container-name>=<image>. Pods are replaced one by one, so the app stays available. Then check progress and the revision history.

```
kubectl set image deployment/my-nginx nginx=nginx:1.25
kubectl rollout status deployment/my-nginx
kubectl rollout history deployment/my-nginx
```

Expected output / observation: 'Waiting for deployment my-nginx rollout to finish: 1 out of 3 new replicas have been updated...' and finally 'deployment "my-nginx" successfully rolled out'. History shows REVISION 1 and 2.

Step 12: Roll back a bad update

If an update fails, return to the previous revision. To practise, deploy a wrong tag and watch the Pods go to ImagePullBackOff, then undo.

```
kubectl set image deployment/my-nginx nginx=nginx:does-not-exist
kubectl get pods
kubectl rollout undo deployment/my-nginx
kubectl rollout status deployment/my-nginx
```

Expected output / observation: After undo the Deployment returns to the working image and all Pods are Running.

Step 13: Expose the Deployment as a Service

A NodePort Service gives a stable address and opens a static port on the node. --port=80 is the Service port. get svc shows the assigned node port (for example 80:31905/TCP).

```
kubectl expose deployment my-nginx --type=NodePort --port=80
kubectl get svc
```

Expected output / observation: service/my-nginx exposed; PORT(S) 80:3xxxx/TCP.

Step 14: Open the Service in the browser

Minikube prints a URL that maps to the NodePort Service. On Windows with the Docker driver keep the terminal open while you use the URL.

```
minikube service my-nginx --url
```

Expected output / observation: A URL such as http://127.0.0.1:54964 - it opens the 'Welcome to nginx!' page.

Step 15: Clean up the resources

Delete the Service, then the Deployment (which also removes its Pods). The last command removes everything in the default namespace (the built-in 'kubernetes' Service is recreated automatically).

```
kubectl delete svc my-nginx
kubectl delete deployment my-nginx
kubectl delete all --all -n default
kubectl get all
```

Expected output / observation: Only 'service/kubernetes ClusterIP' remains.

Step 16: Stop or delete the cluster

stop pauses the cluster (can be started again); delete removes it permanently.

```
minikube stop
minikube delete
```

Problem	Fix
minikube start fails / Docker not found	Start Docker Desktop first; run 'minikube delete' then start again.
Pod stuck in ImagePullBackOff	Wrong image name/tag or no internet - check with kubectl describe pod.
kubectl: command not found	Reopen the terminal after installing, or add kubectl to PATH.
Service URL does not open	Keep the minikube service terminal open (Docker driver on Windows).

Files and content

nginx-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
```

The Deployment and Service in this experiment are created with commands, so no other file is needed.

Quick command reference

```
kubectl create deployment my-nginx --image=nginx
kubectl scale deployment my-nginx --replicas=3
```

```
kubectl set image deployment/my-nginx nginx=nginx:1.25
kubectl expose deployment my-nginx --type=NodePort --port=80
```

Result

Docker, kubectl and Minikube were installed and a local Kubernetes cluster was started. A Pod was created and accessed by port-forwarding, a Deployment was created and scaled to 3 replicas, a rolling update and rollback were performed, the Deployment was exposed as a NodePort Service and opened in the browser, and all resources were cleaned up.

Exp No. 7

Kubernetes Deployment and Service for a Python App from Docker Hub

Deployment (2 replicas) + NodePort Service on Minikube

Aim

To deploy a Python web application stored as a public image on Docker Hub to a Kubernetes (Minikube) cluster using a Deployment and a NodePort Service, and to access the running application through Minikube.

Introduction

In real projects Kubernetes does not build images; it **pulls ready-made images from a registry** such as Docker Hub, AWS ECR or Google Artifact Registry, so the same tested image is promoted unchanged from development to production. Applications are described **declaratively** in YAML manifests - you state the desired state (image, replicas, ports) and Kubernetes continuously makes the real state match it. Manifests can be stored in Git and applied by CI/CD tools, giving repeatable and auditable releases.

Key terms

Term	Meaning
Docker Hub	Public registry from which the cluster nodes pull the image.
Deployment	Creates and maintains the desired number of identical Pods and handles updates.
Replicas	Number of identical Pod copies kept running (2 here) - gives high availability.
Labels / Selector	app: python-web on the Pods; the Deployment's selector.matchLabels and the Service's selector use it to find the Pods.
containerPort	Port the app listens on inside the container (5000 for Flask).
Service (NodePort)	Exposes the Pods on a static port of the node and load-balances across the replicas.
port / targetPort	port = Service port; targetPort = container port that receives the traffic.

Description

A Deployment *python-web-deployment* runs two replicas of a Flask image from Docker Hub listening on port 5000. A NodePort Service *python-web-service* selects the Pods by the label app=python-web and forwards port 5000. The command *minikube service python-web-service --url* returns a URL with which the Flask 'Welcome to Flask + Docker!' page is opened in the browser.

Procedure

Step 1: Start the cluster and verify the node

Docker Desktop must be running.

```
minikube start --driver=docker
minikube status
kubectl get nodes
```

Expected output / observation: Host/kubelet/apiserver Running; node 'minikube' STATUS Ready.

Step 2: Choose and verify the Python image

Use a public Python web image from Docker Hub. The record's text names dockersamples/flaskapp, while your screenshots use your own Exp 2 image klassy01/flask-docker-app; use whichever one you will put in the YAML. Pulling it first confirms the registry is reachable.

```
docker pull klassy01/flask-docker-app
docker images klassy01/flask-docker-app
```

Expected output / observation: 'Status: Image is up to date / Downloaded newer image' and the image is listed with its size.

Step 3: Write the Deployment manifest

Create python-deployment.yaml (see Files section). Key lines: **replicas: 2** (two Pods), **selector.matchLabels app: python-web** (which Pods this Deployment owns), **template.metadata.labels** (must match the selector), **image** (the Docker Hub image) and **containerPort: 5000**.

Step 4: Apply the Deployment and check it

```
kubectl apply -f python-deployment.yaml
kubectl get deployments
kubectl get pods
kubectl describe deployment python-web-deployment
```

Expected output / observation: deployment.apps/python-web-deployment created; READY 2/2, UP-TO-DATE 2, AVAILABLE 2; two Pods named python-web-deployment-xxxxxxxxxx-xxxxx in Running state. describe shows replicas, image, strategy (RollingUpdate) and the Events.

Step 5: Write the Service manifest and apply it

Create python-service.yaml (see Files section). type NodePort exposes the app outside the cluster; the selector sends traffic to the Pods labelled app=python-web; port and targetPort are both 5000.

```
kubectl apply -f python-service.yaml
kubectl get svc
```

Expected output / observation: service/python-web-service created. get svc shows python-web-service NodePort with PORT(S) 5000:30860/TCP (the node port is random in the range 30000-32767) next to the default kubernetes ClusterIP service.

Step 6: Access the application

Ask Minikube for the URL and open it in a browser. With the Docker driver on Windows the terminal must remain open.

```
minikube service python-web-service --url
```

Expected output / observation: A URL like http://127.0.0.1:54964 is printed; the page 'Welcome to Flask + Docker!' with a blue gradient card is displayed.

Note: Optional test of load balancing/self-healing: kubectl delete pod <one-pod-name> - a new Pod is created immediately and the site stays available.

Step 7: Clean up

Delete the Service and the Deployment (the Pods are deleted with it), confirm, and optionally stop or delete the cluster.

```
kubectl delete svc python-web-service
kubectl delete deployment python-web-deployment
kubectl get all
minikube stop
minikube delete
```

Expected output / observation: kubectl get all lists only service/kubernetes ClusterIP 10.96.0.1.

Problem	Fix
Pods in ImagePullBackOff	Image name/tag wrong or repository private - check spelling and that the image is public.
Pods Running but page not opening	Service selector label does not match Pod labels, or targetPort is not 5000.
URL not opening on Windows	Keep the minikube service terminal open; do not press Ctrl+C.
READY 0/2 for long time	Run kubectl describe pod <name> and read the Events section.

Files and content

python-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: python-web-deployment
  labels:
    app: python-web
spec:
  replicas: 2
  selector:
```

```
    matchLabels:
      app: python-web
  template:
    metadata:
      labels:
        app: python-web
    spec:
      containers:
      - name: python-web
        image: klassy01/flask-docker-app:latest
        ports:
        - containerPort: 5000
```

python-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: python-web-service
spec:
  type: NodePort
  selector:
    app: python-web
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
```

Result

A Python web application image from Docker Hub was deployed to a Minikube Kubernetes cluster using a Deployment with two replicas. The Deployment was exposed through a NodePort Service, the application was opened in the browser using the URL given by Minikube, and all resources were cleaned up.